

UCS503P - Software Engineering Project

Cardio AI: Explainable Heart Disease Prediction

Using CNN-LSTM Attention Network with Grad-CAM Interpretability

Submitted by:

1024170434 Dhruv Singla

1024170451 Srishti

Group: BE Third Year, CSE

Submitted to:

Nisha Thakur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.1.1	Problem Statement	2
1.2	Project Objectives	3
2	Methodology and Design	4
2.1	System Architecture	4
2.1.1	High-Level Design and Modeling	4
2.1.2	Technical Stack	4
2.2	Implementation Details	5
2.2.1	Module 1: UI Prototyping and Frontend Architecture	5
2.2.2	Module 2: Deep Learning Baseline Pipeline (CNN)	10
2.3	System Modeling and Project Diagrams	10
2.3.1	Project Scheduling and Timeline (Gantt Chart)	11
2.3.2	UML Use Case Diagram	12
2.3.3	UML Activity Diagram	13
2.3.4	UML Activity Diagram (Swimlanes)	14
2.3.5	UML Sequence Diagram	15
2.3.6	UML Collaboration Diagram	16
3	Progress Evaluation	17
3.1	Current Status and Validation	17
3.2	Analysis of Progress	17
4	Conclusion	18
4.1	Summary of Work	18
4.2	Future Work	18

Chapter 1

Introduction

1.1 Background and Context

Cardiovascular disease remains the leading cause of mortality globally, necessitating advanced, early-prediction systems. While deep learning models can predict heart disease from routine clinical measurements—such as blood pressure, cholesterol levels, and ECG markers—their inherent “black box” nature poses a significant barrier to clinical adoption. Regulatory bodies and medical ethics demand interpretable, auditable predictions. This project, **Cardio AI**, aims to bridge this gap by prioritizing clinical transparency. Our objective is to translate complex deep neural network confidence scores into actionable, human-readable clinical narratives, empowering physicians to trust AI-assisted diagnoses.

1.1.1 Problem Statement

While standard classification tasks can achieve high accuracy on clinical datasets, the primary challenge addressed by this project is shifting the focus from simple heart disease classification to **clinical transparency**—making deep learning explainable rather than a black box. Current challenges in AI-driven heart disease prediction include:

- **Black-Box Predictions:** Deep learning models, while highly accurate, offer no clinical reasoning or transparent decision pathways, leading to physician reluctance.
- **Feature Opacity:** The cardiovascular domain involves heterogeneous clinical features (e.g., numerical vitals, categorical ECG types, binary indicators). Understanding which specific physiological parameters drive a patient’s risk score is non-trivial.
- **Operational Constraints & Overfitting Risks:** Many theoretical models fail in practice due to overfitting on limited datasets (such as the UCI Cleveland dataset). Robust systems must deliver immediate value and incorporate strategies to fail fast through rapid iteration.
- **Lack of Visual Evidence:** Clinicians lack interactive visual artifacts (such as heatmaps or feature importance charts) that map model confidence directly back to patient physiology.

- **Absence of Longitudinal Tracking:** Existing diagnostic tools rarely support patient-history-aware predictions or longitudinal trend analysis across repeated assessments.

1.2 Project Objectives

To address these challenges, we have defined the following core objectives:

1. **Develop an Explainable Deep Learning Model:** Architect and train a hybrid CNN-LSTM-Attention model on the UCI Cleveland Heart Disease dataset. This model will incorporate multiple Explainable AI (XAI) methods, specifically Grad-CAM, Attention weights, and SHAP (Shapley Additive exPlanations), to decode the model's reasoning.
2. **Deliver an Interactive Clinical Dashboard:** Create a responsive, premium Single Page Application (SPA) using HTML5, CSS3, Bootstrap 5, and JavaScript. This dashboard will allow clinicians to input physiological parameters and visualize predictions via Plotly.js interactive charts.
3. **Automate Clinical Reporting:** Generate automated PDF reports featuring AI-driven narrative summaries, feature-by-feature clinical interpretations, and guideline-referenced evaluations.
4. **Implement Patient Management Architecture:** Design a robust backend using Flask (Python) and a MySQL database to securely track longitudinal patient records.

Chapter 2

Methodology and Design

2.1 System Architecture

Our proposed solution combines deep learning engineering with clinical interpretability, focusing on robust pipeline design and multi-method XAI.

2.1.1 High-Level Design and Modeling

During the initial phase of the project, significant effort was dedicated to defining actor modeling, system flow, and architectural blueprints.

- **Deep Learning Pipeline:** The core prediction engine is designed around a CNN-LSTM-Attention network. Clinical data will be transformed into spatiotemporal formats using Gramian Angular Summation Field (GASF) image encoding for robust feature extraction.
- **Multi-Method Explainability Engine:** The architecture maps parallel processing flows to compute SHAP values, Grad-CAM 2D heatmaps, and Attention weights simultaneously after the CNN-LSTM inference.
- **UML Modeling:** We designed a comprehensive UML Use Case Diagram establishing boundaries and main features for the primary actor (Clinician) and secondary actors (Patient, ML Engine, MySQL Database). Features include predicting risk, viewing XAI charts, performing Exploratory Data Analysis (EDA), and downloading PDF reports.
- **Activity Diagram:** A 4-swimlane UML Activity Diagram was produced to clarify responsibilities across the Frontend (Clinician), Middleware (Flask Web Server), Backend (ML Prediction Engine), and Storage (MySQL).

2.1.2 Technical Stack

The project leverages a modern, high-performance technology stack:

- **Frontend Engineering:** HTML5, modern CSS3 (with custom animations), Bootstrap 5, and Vanilla JavaScript.

- **Data Visualization & Aesthetics:** Plotly.js for rendering interactive charts, combined with Three.js and Vanta.js for dynamic, hardware-accelerated cybernetic backgrounds.
- **Audio Feedback:** Web Audio API for synthesizing Sci-Fi diagnostic audio cues without external audio files.
- **Machine Learning:** TensorFlow/Keras for the CNN architecture, scikit-learn for data preprocessing, and pyts for Gramian Angular Summation Field (GASF) transformations.
- **Backend & Integration (Future Work):** Flask (Python) for the RESTful API bridge between the ML engine and the Frontend SPA.

2.2 Implementation Details

2.2.1 Module 1: UI Prototyping and Frontend Architecture

The frontend Single Page Application (SPA) prototype has been successfully implemented, focusing heavily on a premium medical cybernetic aesthetic. The UI was designed to minimize cognitive load while maximizing clinical engagement.

Visual Aesthetics and Theming

We integrated glassmorphism techniques—utilizing semi-transparent backgrounds with backdrop blur filters—to provide tactile feedback on diagnostic panels. A sophisticated dark theme (#050811) was chosen to reduce eye strain for clinicians. The background features an interactive 3D Vanta.js network animation that dynamically shifts colors (from neon blue to danger red) based on the computed patient risk probability. Custom typography utilizes Google Fonts **Outfit** and **Plus Jakarta Sans** for high legibility in clinical settings. Furthermore, bespoke CSS animations such as CRT scanlines and floating 3D depth (`translateZ`) elements enhance the futuristic, premium feel of the dashboard.

Dynamic Input Interfaces

The application architecture relies on three primary SPA states manipulated dynamically via JavaScript:

- **Parameter Input Grid:** A responsive 3-column layout captures 13 critical physiological features (e.g., Age, Sex, Chest Pain Type, Resting BP, Cholesterol, Fasting Blood Sugar, Resting ECG, Max Heart Rate, Exercise Induced Angina, ST Depression, ST Slope, Number of Major Vessels, Thalassemia).
- **Dual-Input Controls & Clinical Ranges:** Features like Age, BP, and Cholesterol utilize synchronized range sliders and numerical inputs. Custom JavaScript logic dynamically updates "Glowing Badges" and clinical reference texts (e.g., tagging BP ≥ 140 as "Hypertension" or Cholesterol ≥ 200 as "Desirable") in real-time as the user adjusts the inputs.

- **Patient Management Mockups:** We implemented UI components for toggling between "New" and "Existing" patients, including an autocomplete datalist structure for searching patient hospital IDs.

Simulated Diagnostic Pipeline

To simulate the backend processing delay, we engineered a highly immersive "System Processing" state:

- **Visual Feedback:** A non-blocking diagnostic scan overlay features complex CSS radar pulse (`radar-pulse`) and sweep animations alongside an SVG sinus rhythm drawing animation.
- **Scrolling Terminal:** A cascading scrolling log terminal outputs simulated computation stages (e.g., "[SYSTEM] Ingesting Telemetry Data...", "[AI] Converting 1D telemetry array to 2D Gramian Field (GASF)...").
- **Audio Synthesis:** We implemented a custom `SciFiAudioController` using the HTML5 Web Audio API. This synthesizes continuous sonar sweeps, data processing ticks, and a success chime upon completion, entirely through code.

Results Dashboard and XAI Visualization

The final SPA state is a comprehensive analytics module designed to present the AI's findings:

- **Risk Probability:** Displays a dynamic progress bar and a CSS-animated beating heart whose pulse rate correlates with the predicted risk percentage.
- **Performance Metrics:** UI containers for displaying Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
- **Plotly.js Integrations:** The dashboard includes initialized Plotly.js containers prepared to render the "XAI Consensus Profile" (comparing Global Attention, Local Grad-CAM, and Patient SHAP weights) and a detailed "2D Grad-CAM Heatmap".
- **AI Narrative Summary:** A dedicated section for generating human-readable clinical narratives based on the model's primary diagnostic drivers.

Client-side HTML5 validations and a robust mock JSON data generator have been configured to ensure the frontend prototype is fully operational and demonstrable prior to backend integration.

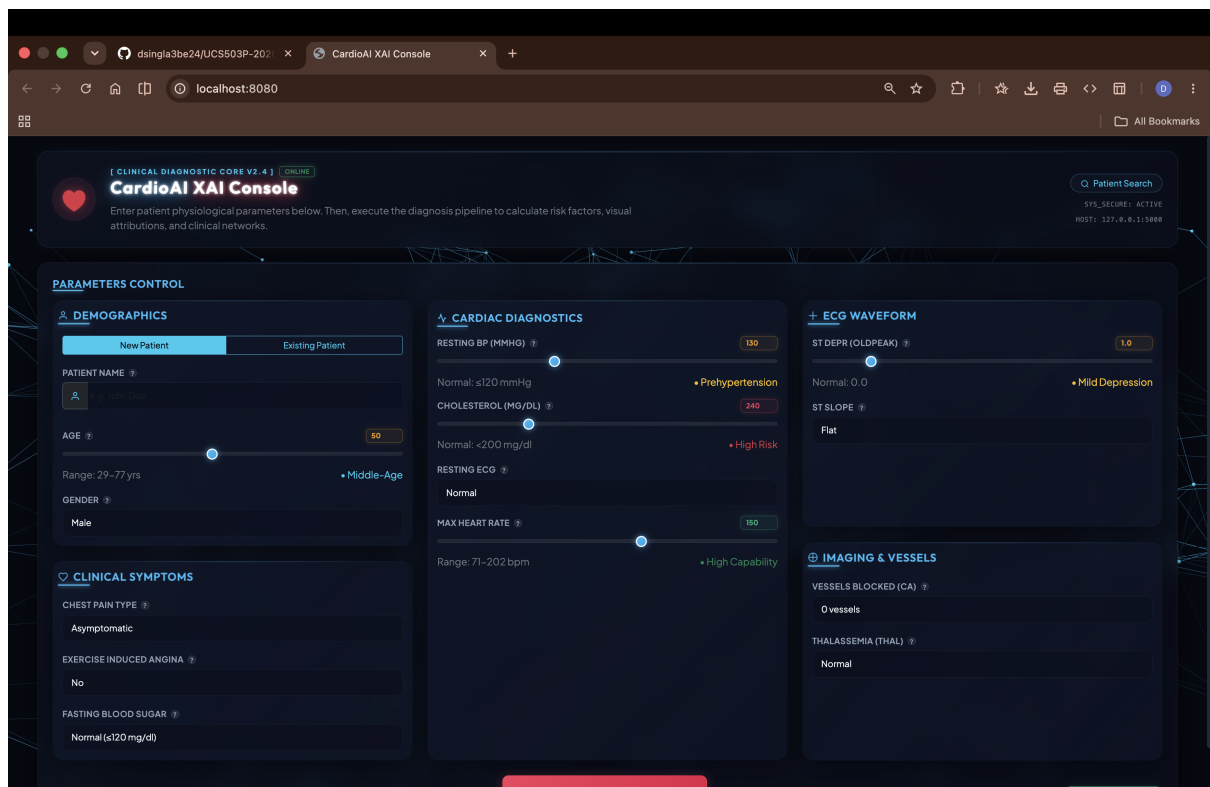


Figure 2.1: Initial Data Input Dashboard with 3D Cybernetic Background.

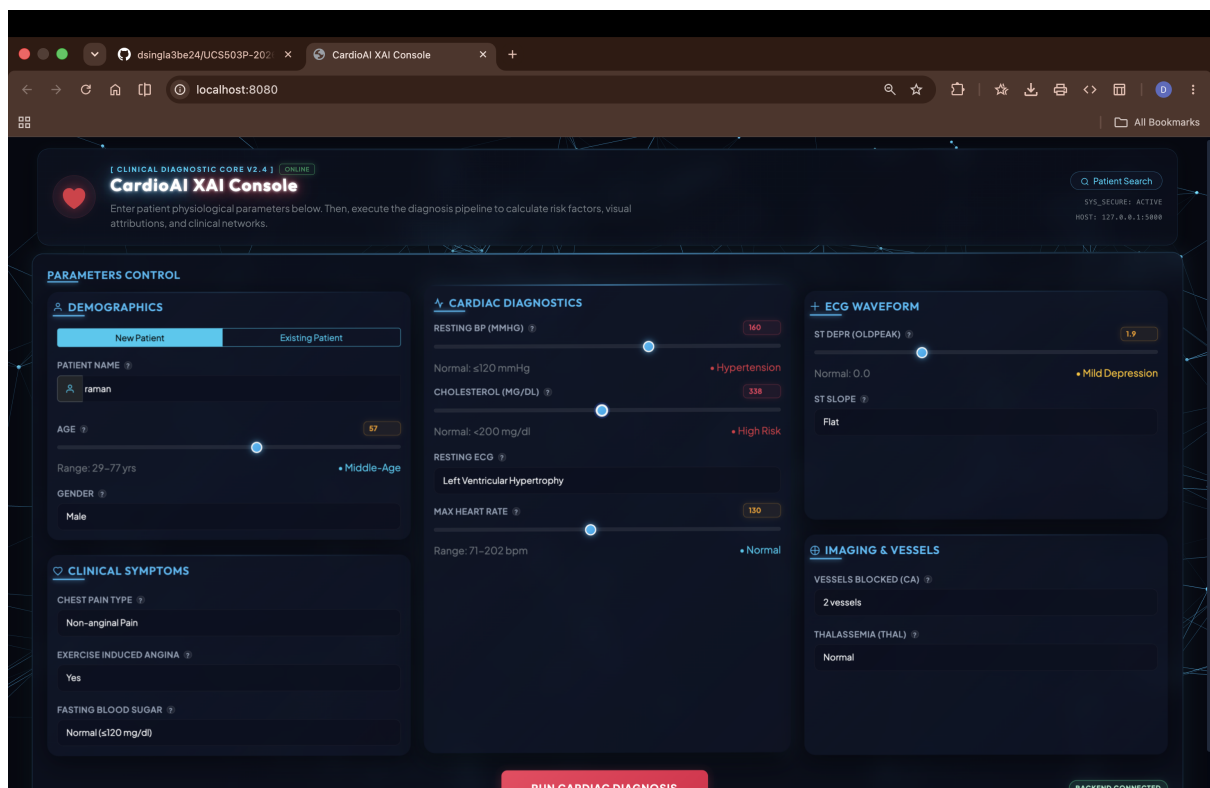


Figure 2.2: Active Parameter Entry utilizing synchronized sliders and glowing clinical badges.

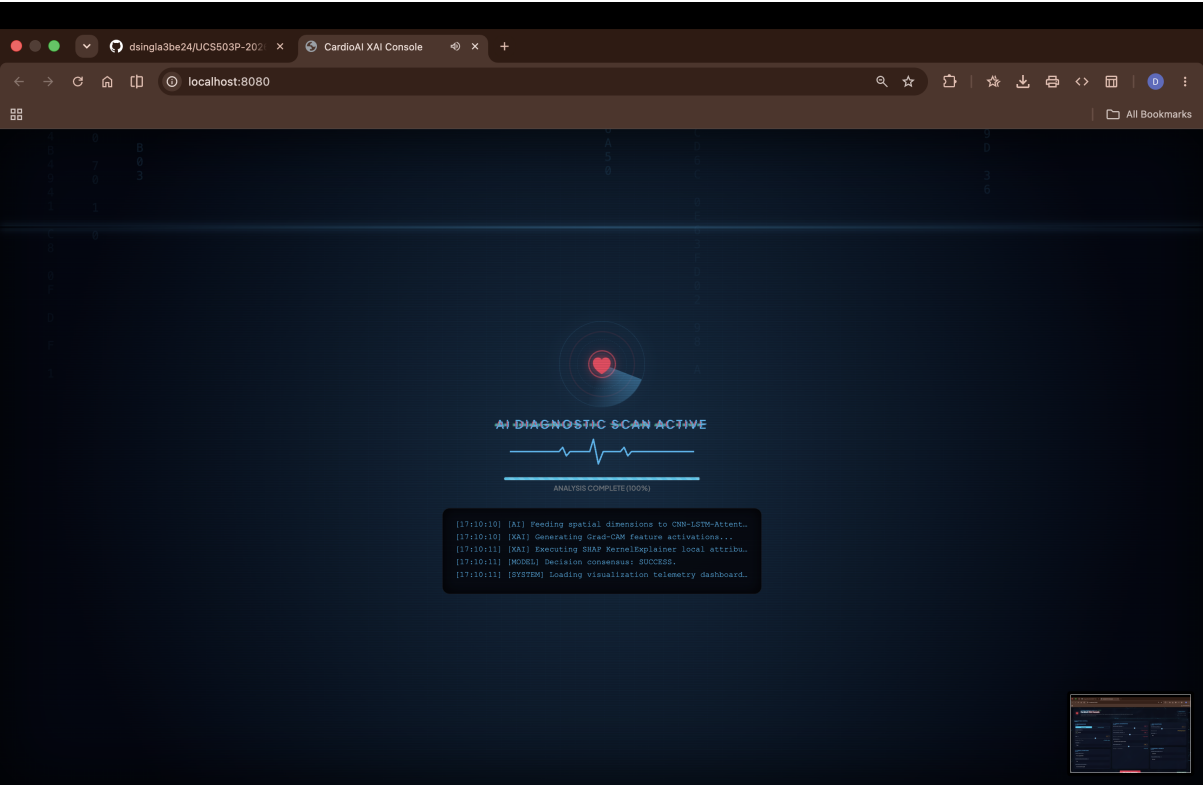


Figure 2.3: System Processing State: Radar sweep and diagnostic overlay.

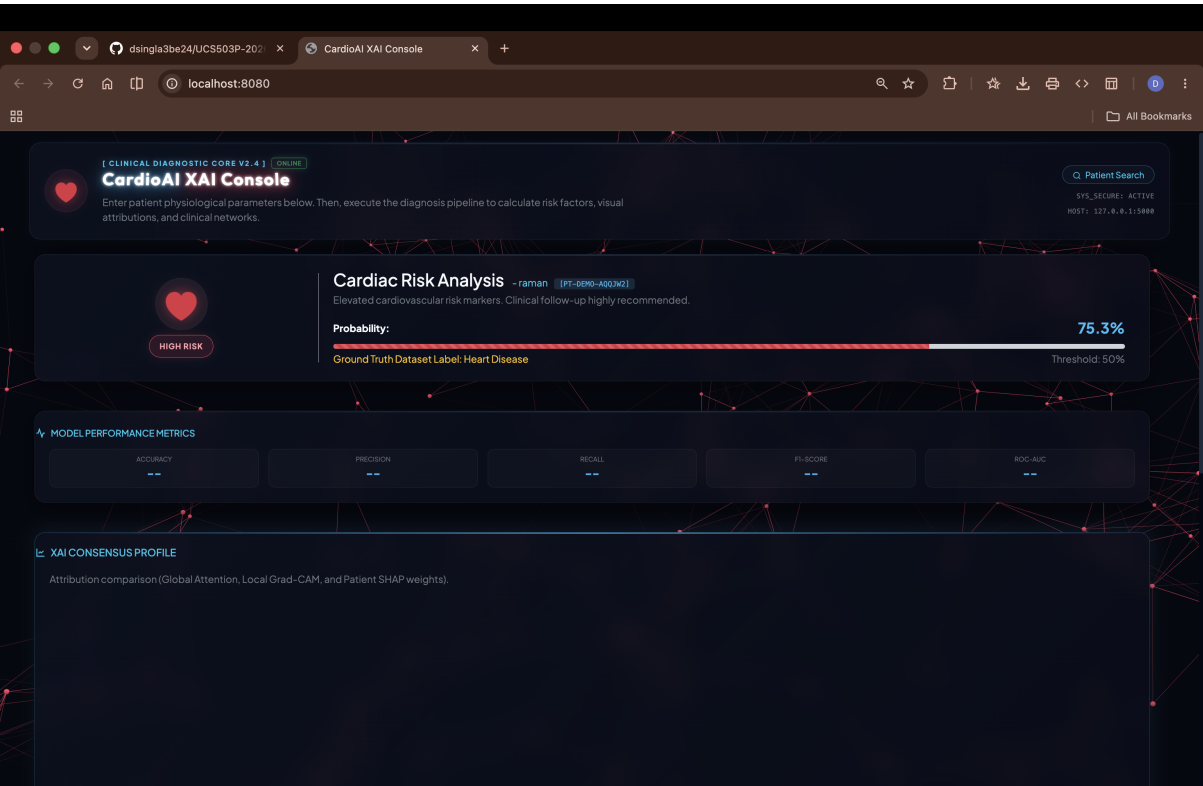


Figure 2.4: Results Dashboard: Predicted Risk Probability and Patient Metrics.

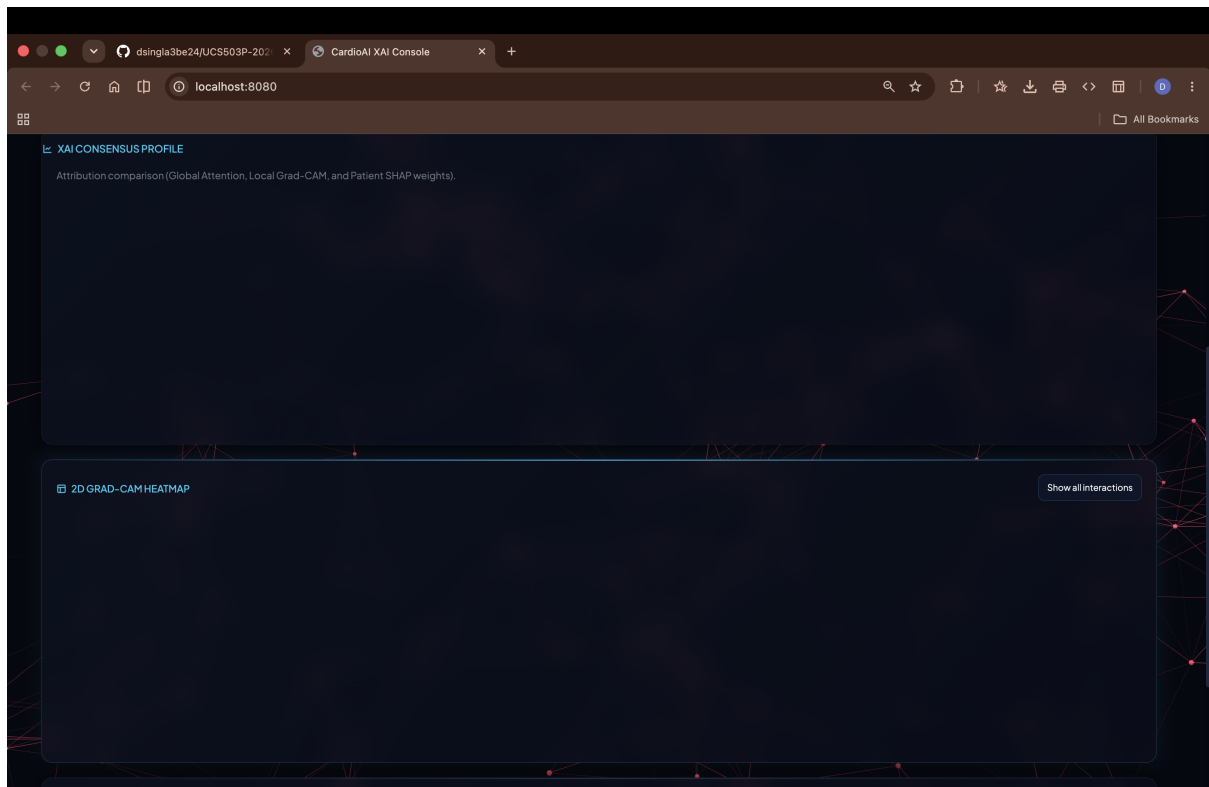


Figure 2.5: XAI Consensus Profile: Comparing Grad-CAM, SHAP, and Attention Heatmaps.

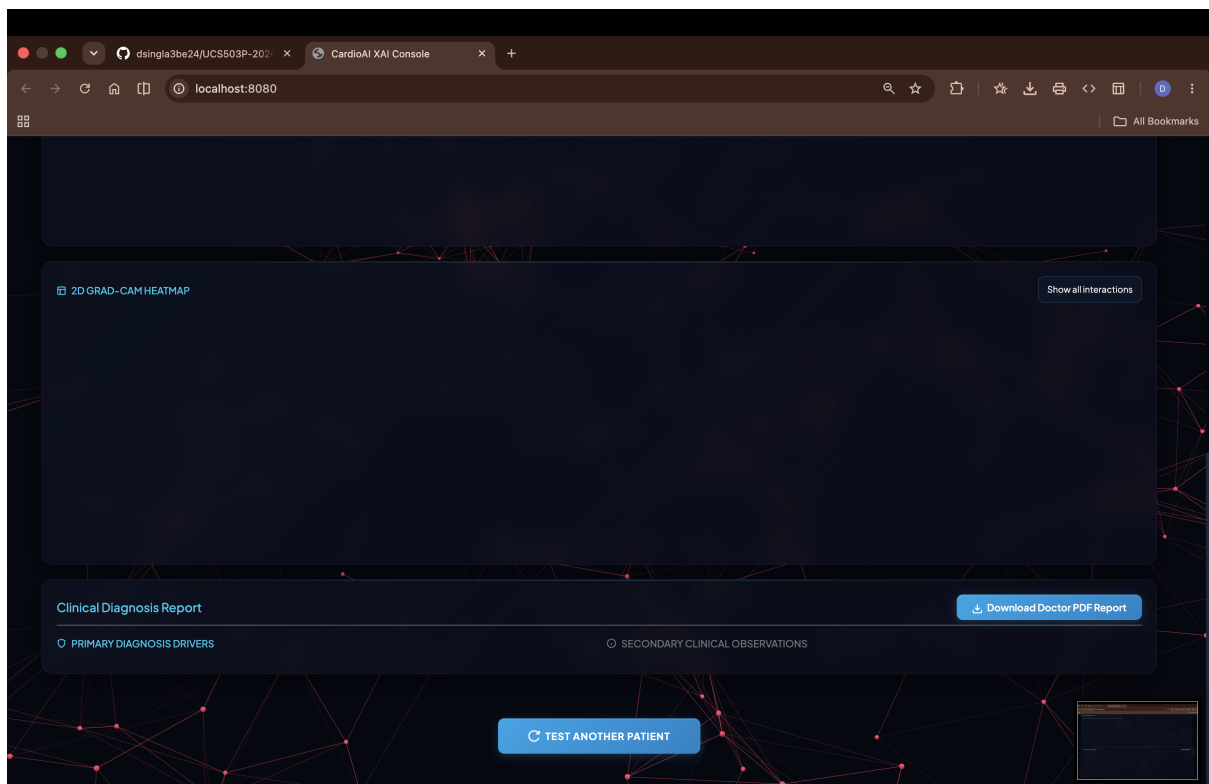


Figure 2.6: Results Dashboard: Patient Narrative Summary and PDF Report Generation.

2.2.2 Module 2: Deep Learning Baseline Pipeline (CNN)

To establish the core prediction engine, we implemented the foundational machine learning backend up to the Convolutional Neural Network (CNN) phase:

- **Data Ingestion & Scaling:** The UCI Heart Disease dataset was ingested and all 13 clinical features were standardized using `MinMaxScaler` to ensure uniform gradient propagation.
- **Mathematical Transformation (GASF):** Since CNNs are optimized for 2D spatial data, we transformed the 1D tabular arrays into 2D Gramian Angular Summation Field (GASF) images. This cutting-edge technique encodes temporal/feature correlations into a visual format.
- **CNN Architecture:** A Keras Sequential model was constructed using `Conv2D` and `MaxPooling2D` layers to extract spatial features from the GASF matrix, culminating in a `Dense` output layer with a Sigmoid activation for binary risk classification.
- **Initial Evaluation:** The baseline CNN was trained successfully, with loss curves and confusion matrices plotted to verify model convergence before the future integration of LSTM and Explainable AI (Grad-CAM/SHAP) components.

2.3 System Modeling and Project Diagrams

To provide a comprehensive blueprint of the Cardio AI system, several architectural and behavioral models were developed.

2.3.1 Project Scheduling and Timeline (Gantt Chart)

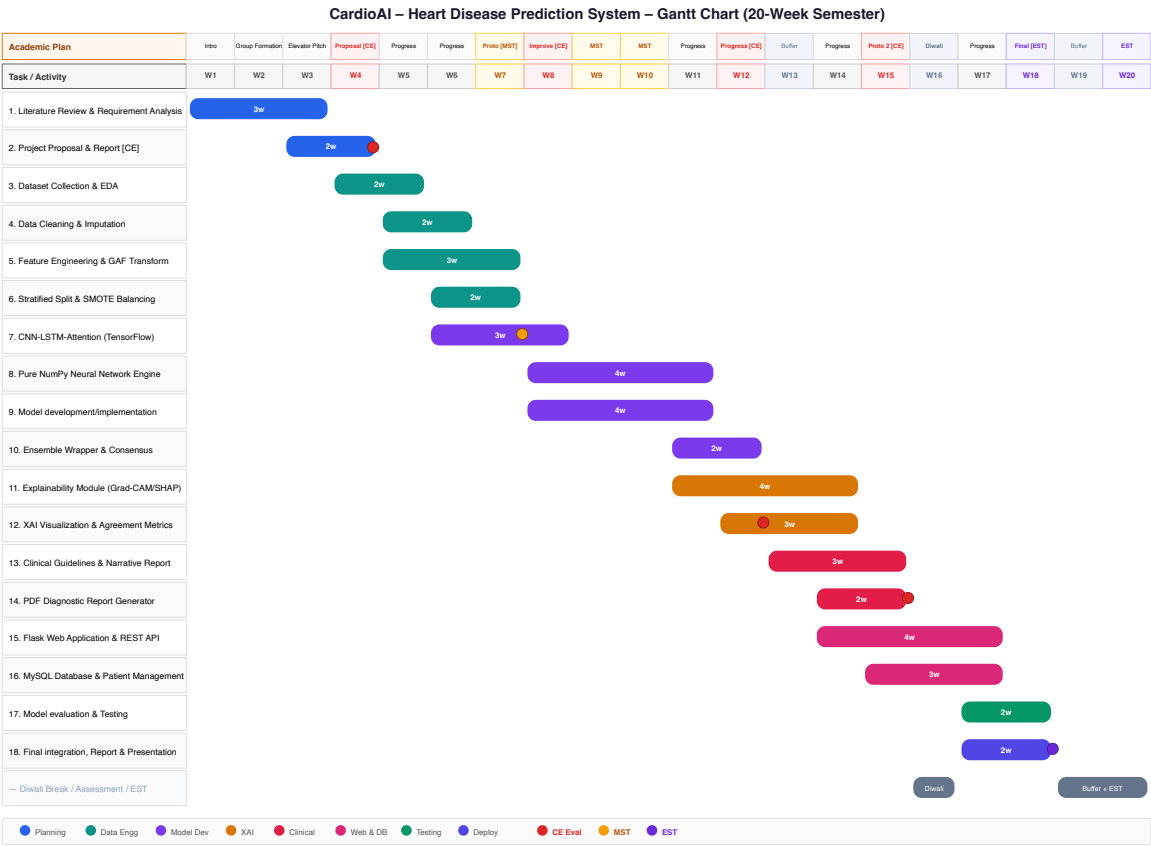


Figure 2.7: Project Scheduling and Timeline (Gantt Chart)

2.3.2 UML Use Case Diagram

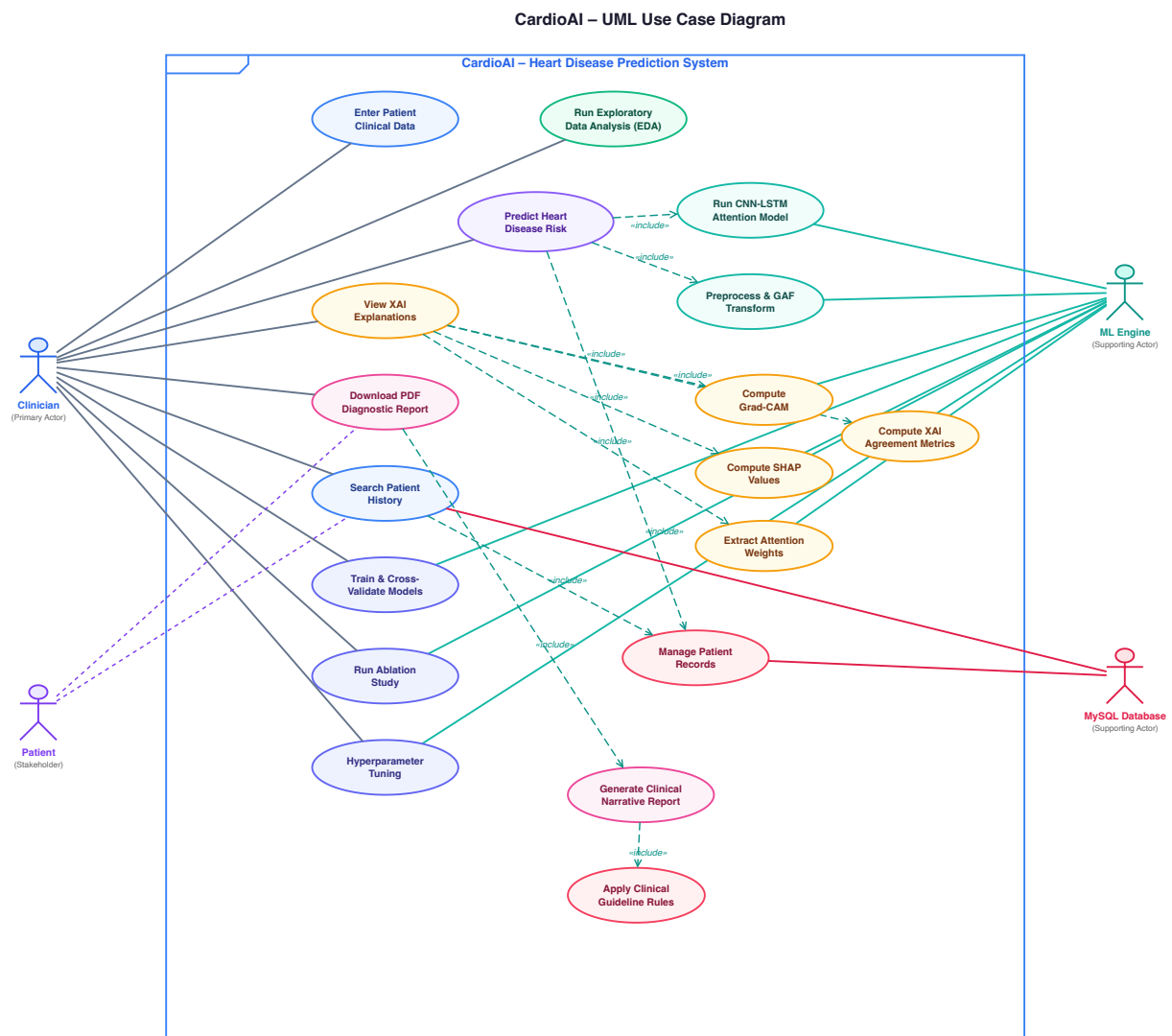


Figure 2.8: UML Use Case Diagram identifying system boundaries and primary actors.

2.3.3 UML Activity Diagram

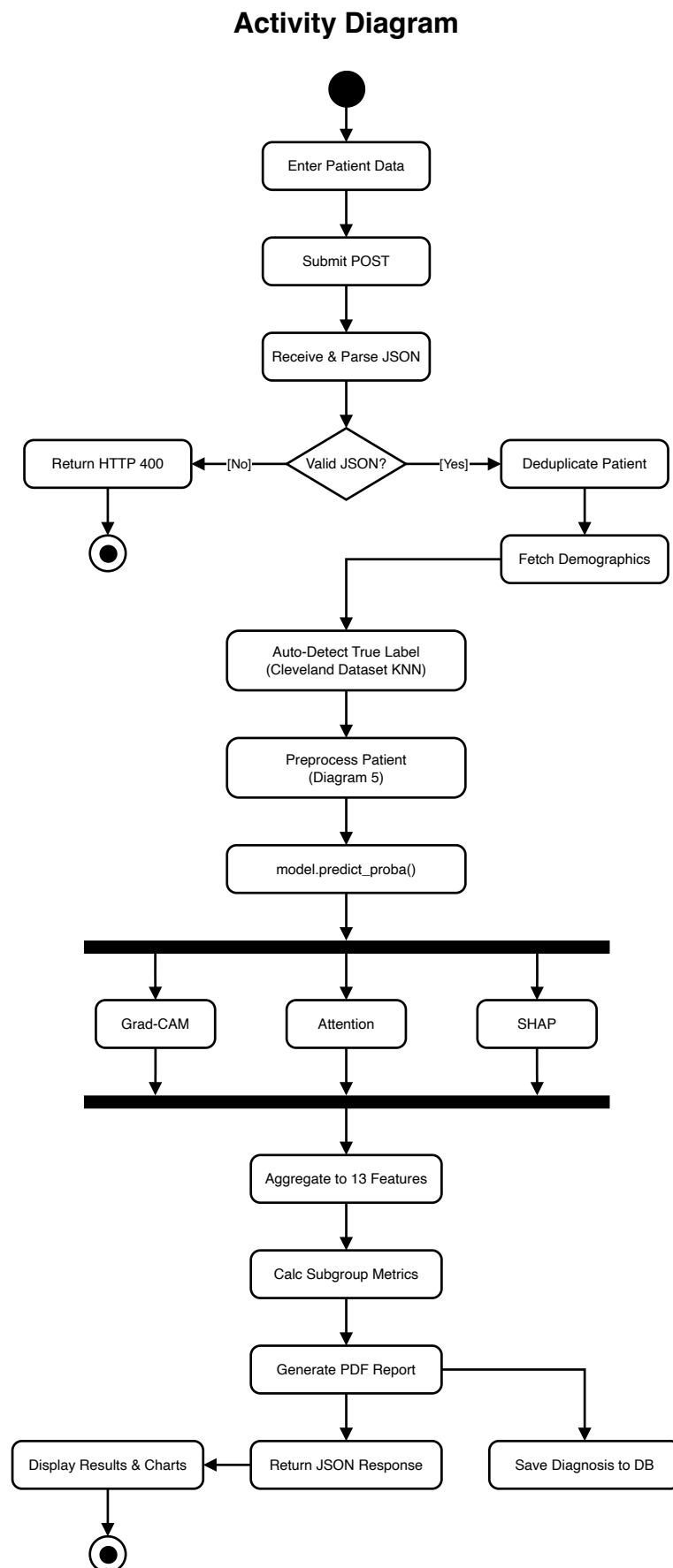


Figure 2.9: UML Activity Diagram mapping the procedural flow of the system.

2.3.4 UML Activity Diagram (Swimlanes)

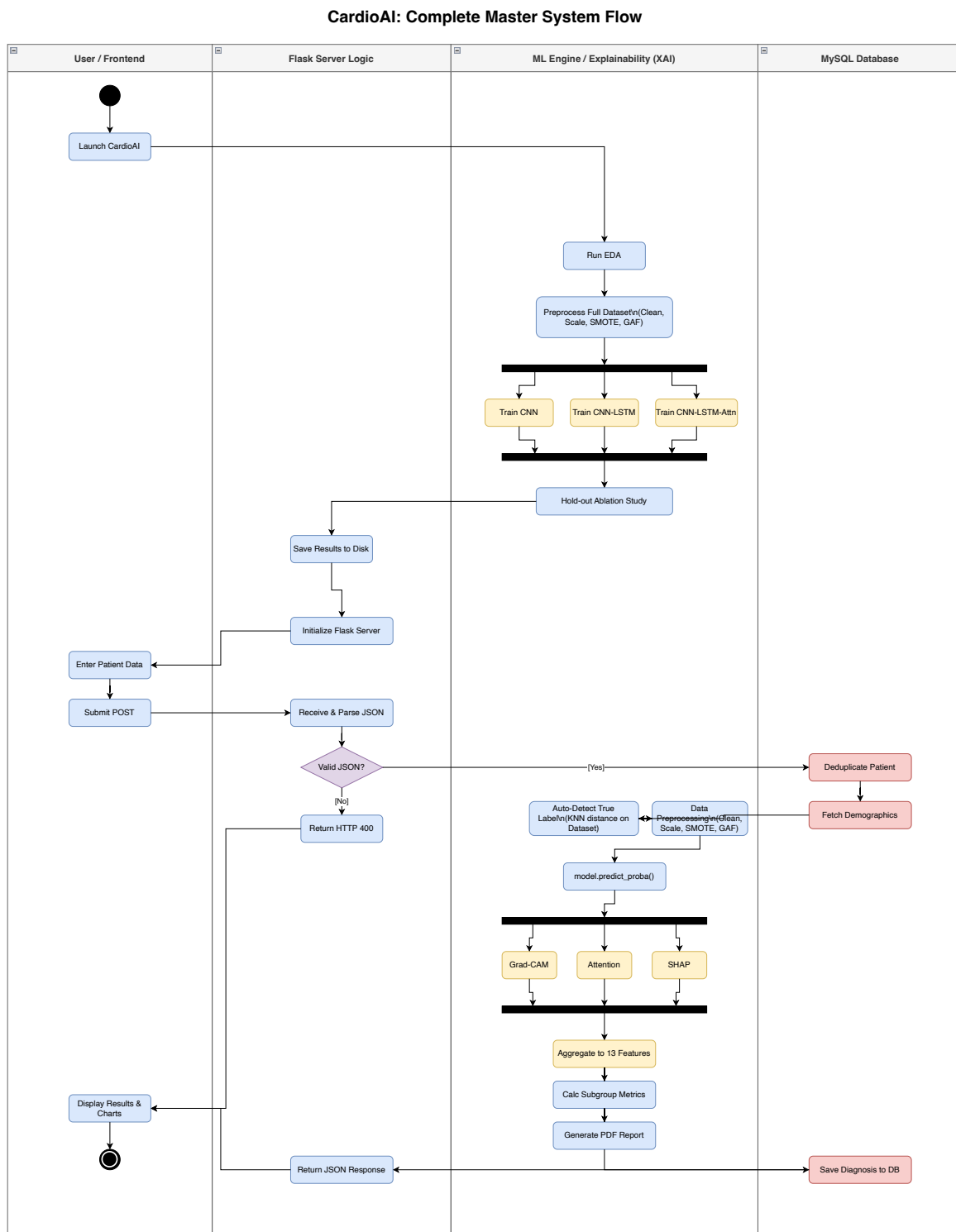


Figure 2.10: UML Activity Diagram mapped across specific actor swimlanes (Frontend, Middleware, Backend, DB).

2.3.5 UML Sequence Diagram

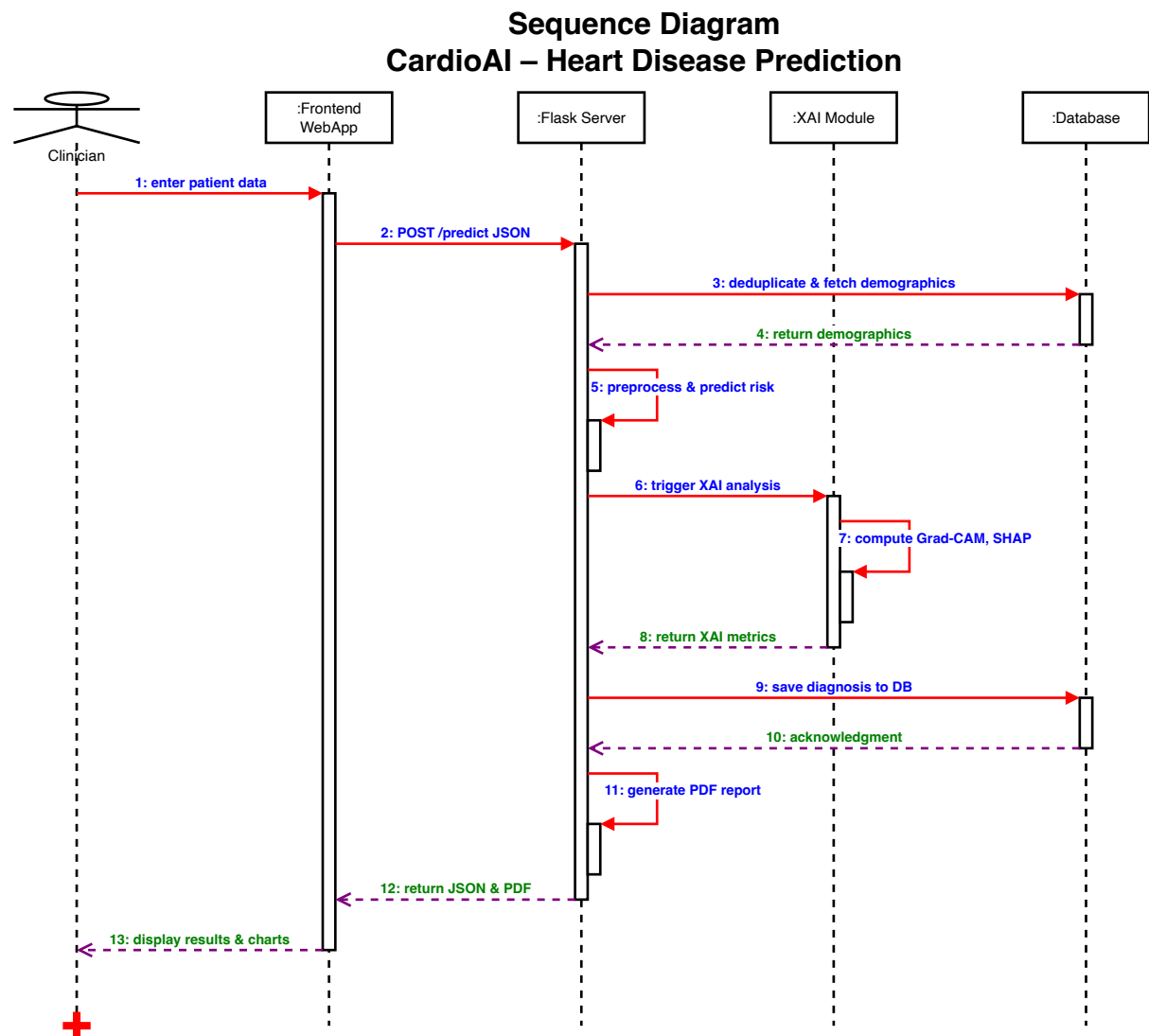


Figure 2.11: UML Sequence Diagram depicting the chronological interaction between system components.

2.3.6 UML Collaboration Diagram

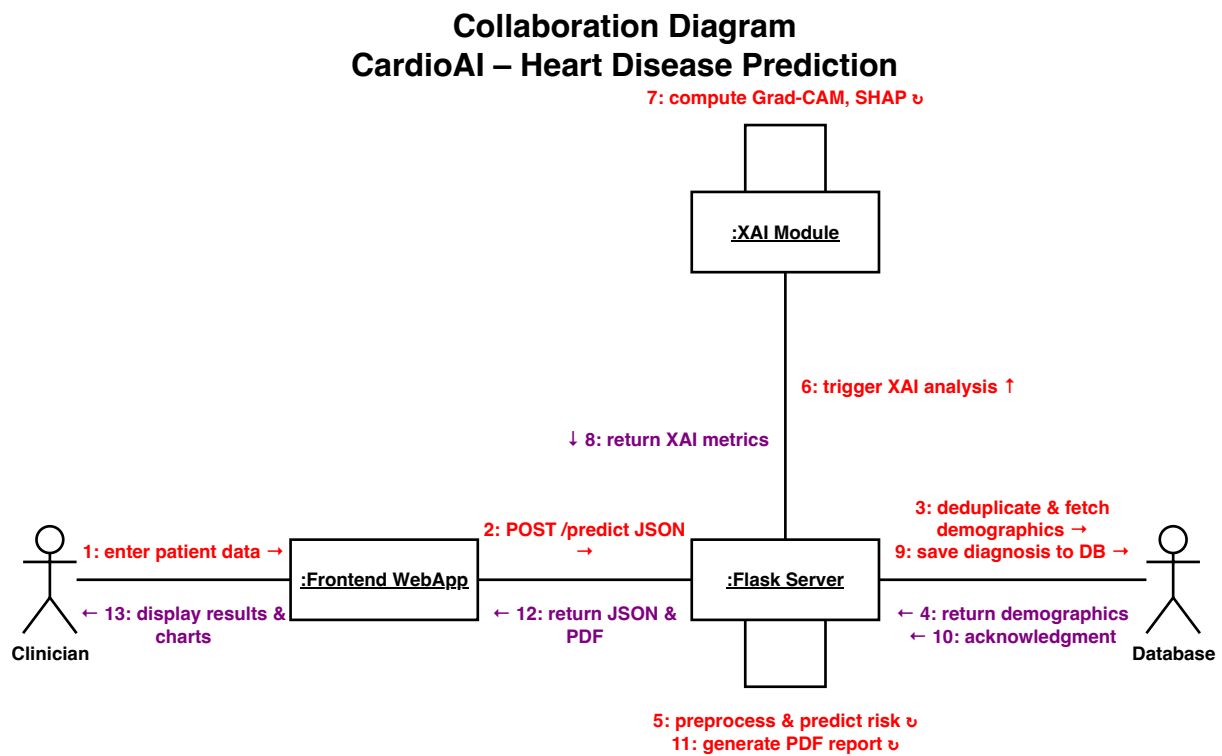


Figure 2.12: UML Collaboration Diagram highlighting the structural communication links between components.

Chapter 3

Progress Evaluation

3.1 Current Status and Validation

As this is the mid-semester evaluation stage, our efforts have successfully culminated in the completion of the foundational architecture, system modeling, a highly polished frontend prototype, and the core CNN machine learning baseline.

- **UML Design Validation:** The comprehensive UML models (Use Case, Activity, Sequence, and Collaboration) have been peer-reviewed and successfully validated as the logical blueprint for our distributed system.
- **UI/UX Testing:** The frontend components have undergone rigorous interface validation. The layout maintains responsiveness across desktop and tablet viewports, and input integrity is enforced via HTML5 attributes and JavaScript event listeners.
- **Mock Integration & Flow:** The SPA transitions seamlessly between data input, processing overlays (with synchronized audio-visual feedback), and the results dashboard. The JavaScript-based mock logic successfully populates the Plotly.js charts and UI badges, proving that the frontend is fully capable of rendering the data structures we intend to send from the backend.
- **CNN Baseline Validation:** The foundational machine learning pipeline was successfully structured and validated in a local Python environment, proving that the clinical dataset can be correctly scaled and transformed into GASF images before being processed by the Keras CNN layers.

3.2 Analysis of Progress

Our progress strictly adheres to our initial Gantt chart objectives for the Requirements, UI, and Baseline ML phases. The robust, state-driven design of the frontend SPA ensures that integrating the Flask REST API will require minimal to no UI refactoring.

Moving forward, the primary engineering challenge will shift to expanding the CNN baseline by adding the LSTM and Attention mechanisms to achieve our target AUC of ≥ 0.85 , and establishing the Flask bridge.

Chapter 4

Conclusion

4.1 Summary of Work

We have established a formidable core foundation for the Cardio AI prediction system. To date, we have:

- Defined the problem scope, focusing on clinical interpretability.
- Architected the proposed ML pipeline and finalized the technical stack.
- Completed comprehensive UML modeling (Use Case, Activity, Sequence, and Collaboration diagrams).
- Developed a detailed project schedule using a Gantt chart.
- Constructed a fully interactive, glassmorphic frontend SPA prototype, complete with dynamic clinical range validations, hardware-accelerated 3D backgrounds, and synthesized audio feedback.
- Engineered the foundational machine learning backend up to the Convolutional Neural Network (CNN) phase, successfully integrating clinical dataset ingestion, MinMaxScaler standardization, and Gramian Angular Summation Field (GASF) image transformations.

4.2 Future Work

Following our established timeline, our immediate next steps encompass expanding the deep learning model and establishing backend integrations:

1. **Advanced Model Training:** Expand the baseline CNN by integrating LSTM and Attention mechanisms to evaluate sequence patterns.
2. **Explainability Engine:** Implement the core Python logic for extracting Grad-CAM heatmaps, Attention weights, and SHAP values.
3. **System Integration:** Deploy the Flask backend to establish the RESTful bridge to our completed frontend SPA.